

PORTADA

- ISTEA — Laboratorio de Minería de Datos
- Informe Técnico — Entrega 3: Monitoreo y Observabilidad
- Predicción de Abandono de Clientes (Churn)
- AndesLink Servicios Digitales S.A.
- Alumnos: Facundo Zubeldia · Gonzalo Martín González Nastovich · Agustín Meza
- Profesor: Ing. Diego Mosquera
- Fecha: Julio 2026

1. Objetivo de la Entrega

En esta tercera y última entrega se implementa la capa de observabilidad del sistema MLOps. El objetivo es monitorear el comportamiento técnico del servicio en producción (latencia, tráfico, errores) y la calidad de los datos que recibe el modelo (drift), para detectar degradación del rendimiento antes de que impacte en el negocio.

2. Arquitectura Completa del Sistema

El sistema final integra 4 servicios orquestados con Docker Compose:

Servicio	Tecnología	Puerto	Función
API de inferencia	FastAPI + Uvicorn	8000	Predicción de churn + métricas
Dashboard GUI	Streamlit	8501	Interfaz para usuarios finales
Métricas	Prometheus	9091	Recolección de métricas técnicas
Visualización	Grafana	3001	Dashboard de monitoreo operativo

Flujo de monitoreo:

1. La API expone métricas en /metrics (formato Prometheus) gracias a prometheus-fastapi-instrumentator
2. Prometheus hace scraping de ese endpoint cada 15 segundos
3. Grafana consulta Prometheus y visualiza las métricas en tiempo real
4. Evidently corre como script separado y genera un reporte HTML de drift comparando datos de referencia vs producción

Facundo Zubeldia:

```
facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$ source /home/facundozubeldia/Documentos/fz_workspace/Lab-Mineria-de-Datos/venv/bin/activate
(fenv) facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$ docker-compose ps
NAME                IMAGE                                COMMAND                  SERVICE    CREATED   STATUS    PORTS
andeslink-api        lab-mineria-de-datos-api            "uvicorn src.api:app..." api         42 minutes ago Up 42 minutes (healthy) 0.0.0.0:8000->8000/tcp, [::]:8000->8000/
andeslink-dashboard  lab-mineria-de-datos-dashboard      "streamlit run src/a..." dashboard  42 minutes ago Up 42 minutes          0.0.0.0:8501->8501/tcp, [::]:8501->8501/
andeslink-grafana    grafana/grafana:latest              "/run.sh"               grafana     42 minutes ago Up 42 minutes          0.0.0.0:3001->3000/tcp, [::]:3001->3000/
andeslink-prometheus prom/prometheus:latest              "/bin/prometheus --c..." prometheus  42 minutes ago Up 42 minutes          0.0.0.0:9091->9090/tcp, [::]:9091->9090/
```

Agustin Meza:

```
PS C:\Users\user\OneDrive\Escritorio\Lab-Mineria-de-Datos> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORT
d5bbffd90a00	grafana/grafana:latest	"/run.sh"	22 minutes ago	Up 21 minutes	0.0.
0.0:3001->3000/tcp, [::]:3001->3000/tcp	andeslink-grafana				
0e63c9acd82b	prom/prometheus:latest	"/bin/prometheus --c..."	22 minutes ago	Up 21 minutes	0.0.
0.0:9091->9090/tcp, [::]:9091->9090/tcp	andeslink-prometheus				
3d0a985928ac	lab-mineria-de-datos-dashboard	"streamlit run src/a..."	22 minutes ago	Up 18 minutes	0.0.
0.0:8501->8501/tcp, [::]:8501->8501/tcp	andeslink-dashboard				
9c3df4bacd5a	lab-mineria-de-datos-api	"uvicorn src.api:app..."	22 minutes ago	Up 22 minutes (healthy)	0.0.
0.0:8000->8000/tcp, [::]:8000->8000/tcp	andeslink-api				

Gonzalo Martín González Nastovich:

```
Windows PowerShell
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

PS C:\Users\ZOOMOSIS NB\Desktop> cd Lab-Mineria-de-Datos
PS C:\Users\ZOOMOSIS NB\Desktop\Lab-Mineria-de-Datos> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
39fb49942e62	grafana/grafana:latest	"/run.sh"	25 minutes ago	Up 25 minutes	0.0.0.0:3001->3000/tcp, [::]:300
1->3000/tcp	andeslink-grafana				
33bae6d1delf	lab-mineria-de-datos-dashboard	"streamlit run src/a..."	25 minutes ago	Up 21 minutes	0.0.0.0:8501->8501/tcp, [::]:850
1->8501/tcp	andeslink-dashboard				
be14d15ab361	prom/prometheus:latest	"/bin/prometheus --c..."	25 minutes ago	Up 25 minutes	0.0.0.0:9091->9090/tcp, [::]:909
1->9090/tcp	andeslink-prometheus				
30b9b3289c6a	lab-mineria-de-datos-api	"uvicorn src.api:app..."	25 minutes ago	Up 25 minutes (healthy)	0.0.0.0:8000->8000/tcp, [::]:800
0->8000/tcp	andeslink-api				

```
PS C:\Users\ZOOMOSIS NB\Desktop\Lab-Mineria-de-Datos> |
```

3. Mejoras incorporadas respecto a la Entrega 2

3.1 Endpoint /health real

Se reemplazó el health check básico por un endpoint que valida que el modelo está cargado y puede realizar una predicción de prueba. El healthcheck de Docker apunta a este endpoint, garantizando que el dashboard solo arranque cuando la API está realmente operativa.

3.2 Validación estricta de categorías

Se reemplazó str por Literal de Python para las variables categóricas (contract_type, payment_method, internet_service, region). Ahora la API devuelve error 422 automáticamente si se envía un valor fuera del dominio permitido, sin necesidad de código adicional.

3.3 Logging estructurado

Se agregó logging con formato timestamp para registrar cada predicción realizada, incluyendo probabilidad de churn y nivel de riesgo. Esto permite auditoría de las inferencias en producción.

3.4 Tests ampliados a 11

Se agregaron tests de categorías inválidas, endpoint /health, payload vacío y umbrales de negocio más exigentes (< 0.5 y > 0.5 en vez de umbrales triviales).

4. Monitoreo Técnico con Prometheus y Grafana

4.1 Métricas recolectadas

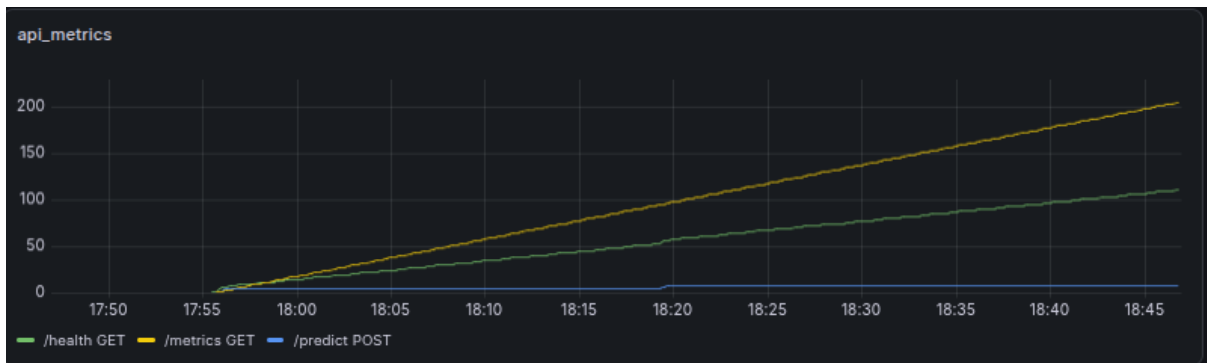
La librería prometheus-fastapi-instrumentator instrumenta automáticamente la API y expone las siguientes métricas en /metrics:

- http_requests_total — contador de requests por endpoint y método HTTP
- http_request_duration_seconds — histograma de latencia por endpoint
- process_cpu_seconds_total — uso de CPU del proceso
- process_resident_memory_bytes — uso de memoria RAM

4.2 Dashboard de Grafana

Se configuró un dashboard con 4 paneles:

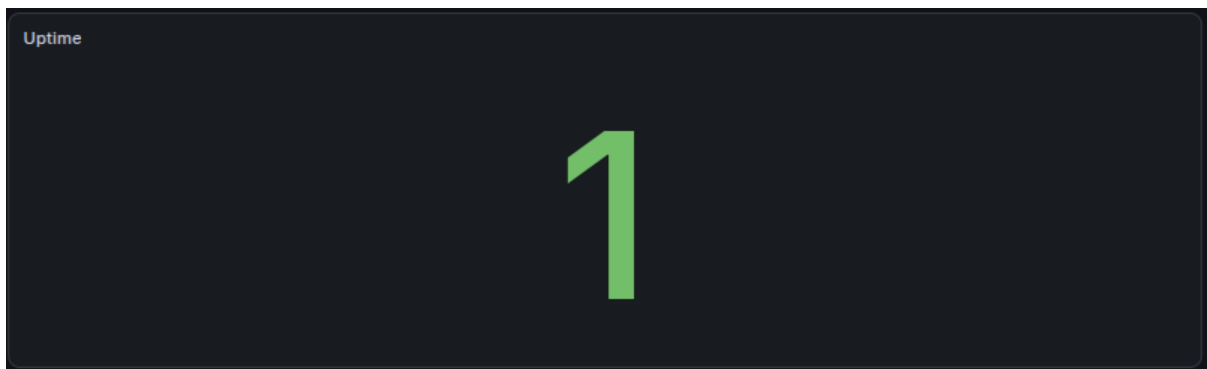
Panel 1 — Tráfico por endpoint: muestra el volumen acumulado de requests a /health, /metrics y /predict a lo largo del tiempo.



Panel 2 — Status codes: muestra la distribución de respuestas HTTP. Todos los responses son 2xx, confirmando que la API responde correctamente.



Panel 3 — Uptime: muestra el tiempo de actividad del servicio desde el último reinicio.



Panel 4 — Latencia del /predict: muestra el tiempo promedio de inferencia del endpoint de predicción. La latencia observada es ~0.015 segundos, lo cual es excelente para inferencia en tiempo real.



4.3 Dashboard de Grafana

Dashboard de Grafana incluido como código en el repositorio mediante provisioning automático — cualquier usuario que levante Docker ve el dashboard sin configuración adicional



5. Monitoreo de Calidad de Datos con Evidently

5.1 ¿Qué es el drift de datos?

El drift ocurre cuando los datos que llegan en producción son estadísticamente diferentes a los datos con los que se entrenó el modelo. Si el perfil de los clientes cambia significativamente, el modelo puede empezar a predecir mal sin que haya ningún error técnico visible.

5.2 Metodología

Se comparó el dataset de entrenamiento (5.000 registros, dataset de referencia) contra datos de producción simulados (1.000 registros) con un perfil de cliente diferente: clientes más nuevos, con cargos más altos y más tickets de soporte.

5.3 Resultados

Evidently detectó drift en **10 de 15 columnas (66.7%)**.

Variable	Tipo	Drift Score	Método
support_tickets	numérica	2.991	Wasserstein distance
monthly_charge	numérica	2.734	Wasserstein distance
late_payments	numérica	1.605	Wasserstein distance
tenure_months	numérica	1.278	Wasserstein distance
avg_monthly_usage_gb	numérica	0.800	Wasserstein distance
total_charges	numérica	0.532	Wasserstein distance
internet_service	categorica	0.206	Jensen-Shannon distance
contract_type	categorica	0.163	Jensen-Shannon distance
payment_method	categorica	0.153	Jensen-Shannon distance
has_streaming	numérica	0.121	Jensen-Shannon distance





Drift is detected for 66.667% of columns (10 out of 15).

Column	Type	Reference Distribution	Current Distribution	Data Drift	Stat Test	Drift Score
> support_tickets	num			Detected	Wasserstein distance (normed)	2.991283
> monthly_charge	num			Detected	Wasserstein distance (normed)	2.734263
> late_payments	num			Detected	Wasserstein distance (normed)	1.604654
> tenure_months	num			Detected	Wasserstein distance (normed)	1.278174
> avg_monthly_usage_gb	num			Detected	Wasserstein distance (normed)	0.800342
> total_charges	num			Detected	Wasserstein distance (normed)	0.531847
> internet_service	cat			Detected	Jensen-Shannon distance	0.20638
> contract_type	cat			Detected	Jensen-Shannon distance	0.163069
> payment_method	cat			Detected	Jensen-Shannon distance	0.152838
> has_streaming	num			Detected	Jensen-Shannon distance	0.120837

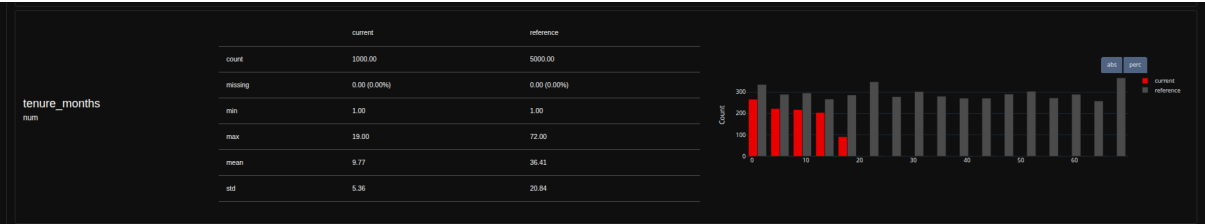
Rows per page: 10 rows

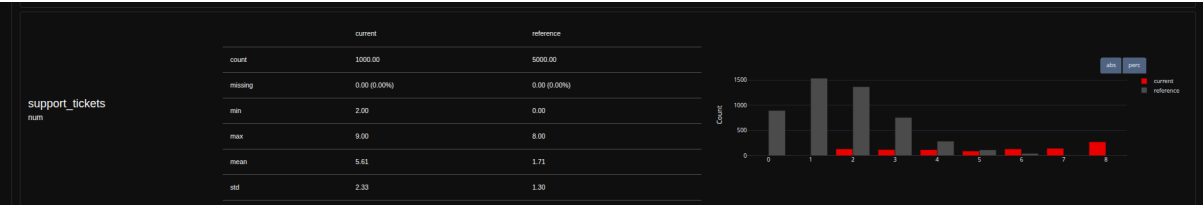
< 1-10 of 15 >

5.4 Interpretación del negocio

Las variables con mayor drift (support_tickets, monthly_charge, tenure_months) son también las más correlacionadas con el churn según el EDA de la Entrega 1. Esto indica que los clientes en producción tienen un perfil de mayor riesgo que los del entrenamiento, lo que podría causar que el modelo subestime la probabilidad de churn real.

Acción recomendada: reentrenar el modelo incorporando los datos de producción para recalibrar la distribución esperada.





6. Tests Automatizados

Se expandió la suite de tests de 7 a 11 casos, incorporando las mejoras técnicas de la entrega 3:

Test	Descripción	Resultado
test_root	GET / devuelve 200	PASSED
test_health_check	GET /health valida modelo cargado	PASSED
test_prediccion_cliente_valido	POST /predict con datos válidos	PASSED
test_prediccion_payload_incompleto	Payload sin campos devuelve 422	PASSED
test_prediccion_tipo_incorrecto	Tipos incorrectos devuelven 422	PASSED
test_categoria_invalida_contract_type	"trimestral" devuelve 422	PASSED
test_categoria_invalida_internet_service	"satelital" devuelve 422	PASSED
test_formato_respuesta	Campos y tipos correctos en response	PASSED
test_prediccion_cliente_bajo_riesgo	Cliente fiel tiene prob < 0.5	PASSED
test_prediccion_cliente_alto_riesgo	Cliente en riesgo tiene prob > 0.5	PASSED
test_payload_vacio	Payload vacío devuelve 422	PASSED

```
(venv) facundozubeldia@pop-os:~/Documentos/fz_workspace/Lab-Mineria-de-Datos$ pytest tests/ -v --html=reports/test_report.html --self-contained-html
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.3, pluggy-1.6.0 -- /home/facundozubeldia/Documentos/fz_workspace/Lab-Mineria-de-Datos/venv/bin/python3.12
cachedir: .pytest_cache
metadata: {'Python': '3.12.3', 'Platform': 'Linux-7.0.11-76070011-generic-x86_64-with-glibc2.39', 'Packages': {'pytest': '9.0.3', 'pluggy': '1.6.0'}, 'Plugins': {'Faker': '40.28.1', 'metadata': '3.1.1', 'anyio': '4.13.0', 'html': '4.2.0', 'hydra-core': '1.3.2'}}
rootdir: /home/facundozubeldia/Documentos/fz_workspace/Lab-Mineria-de-Datos
plugins: Faker-40.28.1, metadata-3.1.1, anyio-4.13.0, html-4.2.0, hydra-core-1.3.2
collected 11 items

tests/test_api.py::test_root PASSED [ 9%]
tests/test_api.py::test_health_check PASSED [ 18%]
tests/test_api.py::test_prediccion_cliente_valido PASSED [ 27%]
tests/test_api.py::test_prediccion_payload_incompleto PASSED [ 36%]
tests/test_api.py::test_prediccion_tipo_incorrecto PASSED [ 45%]
tests/test_api.py::test_categoria_invalida_contract_type PASSED [ 54%]
tests/test_api.py::test_categoria_invalida_internet_service PASSED [ 63%]
tests/test_api.py::test_formato_respuesta PASSED [ 72%]
tests/test_api.py::test_prediccion_cliente_bajo_riesgo PASSED [ 81%]
tests/test_api.py::test_prediccion_cliente_alto_riesgo PASSED [ 90%]
tests/test_api.py::test_payload_vacio PASSED [100%]

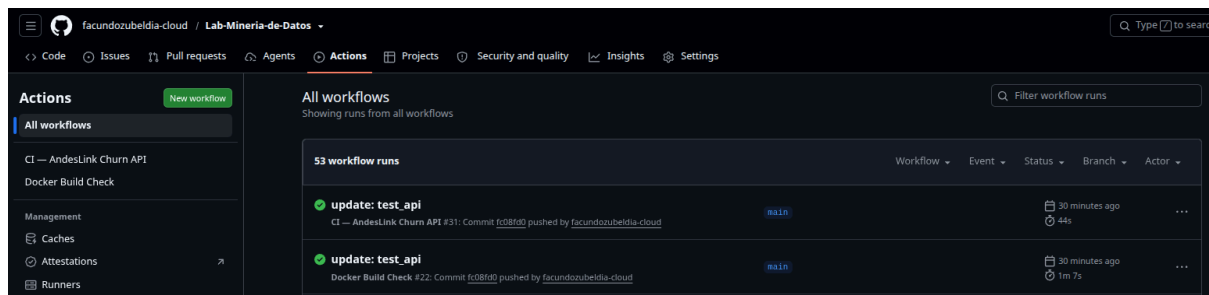
----- Generated html report: file:///home/facundozubeldia/Documentos/fz_workspace/Lab-Mineria-de-Datos/reports/test_report.html -----
11 passed in 1.64s
```

7. GitHub Actions — CI/CD

Se mantienen dos workflows automáticos que se ejecutan con cada git push:

ci.yml — corre los 11 tests con pytest. Si alguno falla, GitHub muestra  y bloquea el merge.

docker-build.yml — construye ambas imágenes Docker y verifica que la API arranca correctamente. Detecta si alguien olvida agregar una dependencia al requirements-docker.txt.



8. Conclusiones Finales

Se completó el ciclo completo de MLOps para AndesLink:

- **Entrega 1:** EDA, preparación de datos, comparación de modelos, serialización, MLflow y DVC
- **Entrega 2:** API de inferencia, dashboard Streamlit, Docker Compose, tests automatizados
- **Entrega 3:** Prometheus, Grafana, Evidently, mejoras técnicas de la API

El sistema cumple con todos los requisitos de observabilidad: monitoreo técnico del servicio (latencia, tráfico, errores) y monitoreo de calidad de datos (drift). La detección de drift en 66.7% de las variables confirma que el sistema es capaz de identificar cuándo el modelo necesita ser reentrenado.

Limitaciones y trabajo futuro:

- El dataset es sintético y los datos de producción son simulados; en producción real se registrarían los requests reales al endpoint /predict
- Sin autenticación en la API; en producción se implementaría OAuth2 o API keys
- El reentrenamiento es manual; en un sistema productivo se automatizaría con Airflow o similar cuando el drift supere un umbral definido
- El remote de DVC es local; en producción apuntaría a S3 o GCS

Desafíos Técnicos y Resolución de Problemas

Durante el desarrollo del proyecto, enfrentamos y resolvimos las siguientes complicaciones críticas:

1. **Gestión de Entornos (Dependency Hell):** Al combinar librerías de entrenamiento (MLflow, Evidently) con las de despliegue (FastAPI, Streamlit), surgieron conflictos de versiones. **Resolución:** Implementamos archivos de requerimientos separados (**requirements.txt** para desarrollo y **requirements-docker.txt** para producción), asegurando imágenes Docker ligeras y estables.
2. **Conflictos de Puertos:** En entornos locales, el puerto **9090** de Prometheus frecuentemente entraba en conflicto con servicios locales. **Resolución:** Implementamos mapeo de puertos dinámico en **docker-compose.yml**.
3. **Persistencia en Grafana:** Los dashboards configurados manualmente se perdían al reiniciar Docker. **Resolución:** Implementamos el aprovisionamiento automatizado mediante archivos

JSON y configuración YAML de Grafana para que el dashboard sea "código" y se levante automáticamente.

4. **Evolución de APIs:** Al migrar entre versiones de Evidently (0.6.7), encontramos cambios en la sintaxis de **Report**. **Resolución:** Ajuste del flujo de ejecución del objeto **Report** a la versión actual de la librería, asegurando la compatibilidad.